

Backend Project Phase-1 Report

University Name: Atria University

Department: Digital transformation

Course: Backend Development – Node.js, Express, MongoDB

Project Title: MinorMap+ : Smart Minor Eligibility & Recommendation Engine

Submitted by: Team 500 OK

Team Lead: Ibrahim

Date: November 11, 2025

1. Abstract

MinorMap+ is a backend-driven academic recommendation system that assists university students in identifying which minor programs best match their strengths, interests, and aptitude. The system enables students to register, log in, complete a quiz, and receive personalized results categorized as **Eligible**, **Potential**, or **Not Recommended** for each available minor.

Built with **Node.js**, **Express.js**, and **MongoDB**, the backend integrates **JWT-based authentication** for secure user access. A **weighted scoring algorithm** computes eligibility by analyzing quiz responses against predefined weights for each minor. Admins can manage questions, weights, and minor data, while students view transparent and interpretable outcomes.

The expected outcome of MinorMap+ is a reliable, data-driven recommendation platform that enhances decision-making, improves academic alignment, and simplifies minor selection for students and advisors alike.

2. Introduction

Choosing a suitable academic minor often confuses students due to a lack of structured guidance. Most rely on peer opinions or assumptions, leading to misaligned choices that affect motivation and career direction. MinorMap+ solves this by implementing an intelligent backend system that evaluates each student's aptitude objectively through a quiz-based scoring engine.

Using a **Node.js + Express.js** backend and **MongoDB** for data storage, the system processes quiz results via a dynamic algorithm that maps responses to multiple minor tracks. This ensures consistent, unbiased, and scalable evaluation.

In today's academic landscape, where personalization and data-driven decision-making are essential, MinorMap+ stands out by offering **secure authentication**, **algorithmic evaluation**, and

clear analytics for both students and advisors. The backend structure promotes modularity, scalability, and real-time data processing—key elements in modern digital education systems.

3. Literature Review

Existing System / Research	Methodology	Limitation	Improvement in MinorMap+
University counseling portals	Manual interviews and static aptitude tests	Subjective, time-consuming	Automated evaluation and consistent scoring logic
Career quiz websites (e.g., 123Test, Truity)	Basic question-answer tests with preset outputs	Lacks transparency, not domain-specific	Academic-focused weights for university minors
Learning Management Systems (LMS)	Limited analytics based on course grades	Ignores personal interest and trait assessment	Combines behavioral inputs with performance logic
AI career recommender prototypes	ML-based prediction models	Complex deployment, needs large datasets	Lightweight backend algorithm suitable for small datasets

MinorMap+ bridges these gaps through a **weighted, interpretable scoring algorithm** that delivers instant, explainable recommendations without ML complexity.

4. System Analysis

Existing System

Most universities rely on **manual counseling** or basic forms to assess students' interest in minors. This lacks objectivity and scalability. Data is often inconsistent, feedback is delayed, and decision quality depends on counselor experience.

Proposed System

MinorMap+ introduces a **backend-controlled recommendation engine**. It evaluates quiz responses, computes scores per minor, and returns ranked recommendations.

Core backend features:

- JWT-secured authentication
- Weighted scoring algorithm
- Role-based admin access
- RESTful APIs for integration

This system ensures data reliability, faster evaluation, and improved decision transparency for all stakeholders.

5. System Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB for Node modules and database
- Internet connection for API testing

Software Requirements

- **Node.js ($\geq 18.x$)**
- **Express.js Framework**
- **MongoDB Compass / Atlas**
- **VS Code, Postman, GitHub**

Functional Requirements

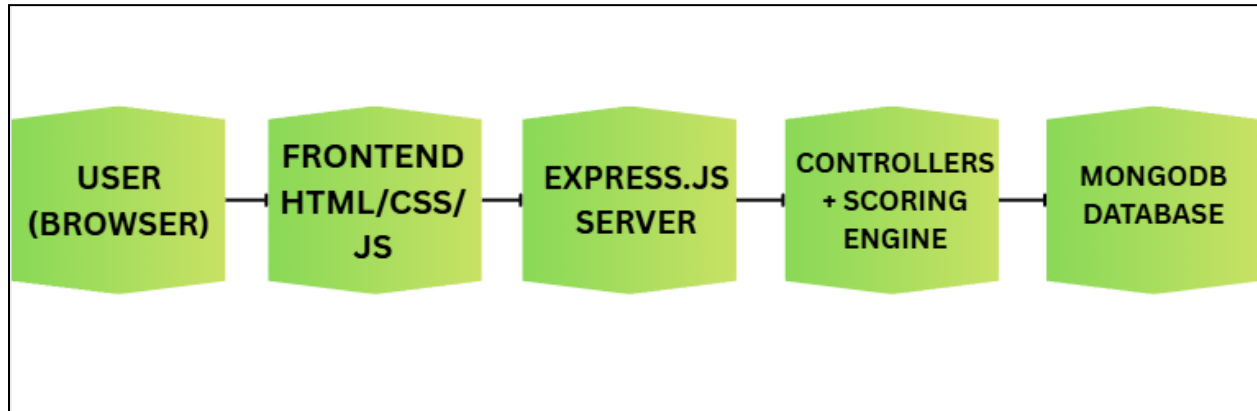
- User Registration and Login
- JWT Authentication & Authorization
- CRUD operations for Questions, Minors, and Results
- Quiz Submission & Eligibility Computation
- Admin Dashboard for Data Management

Non-Functional Requirements

- **Security:** Password hashing (bcrypt), JWT token validation
- **Scalability:** Modular Node.js architecture
- **Reliability:** Consistent API responses with error handling
- **Maintainability:** MVC pattern and environment variables (.env)

6. System Design & Architecture

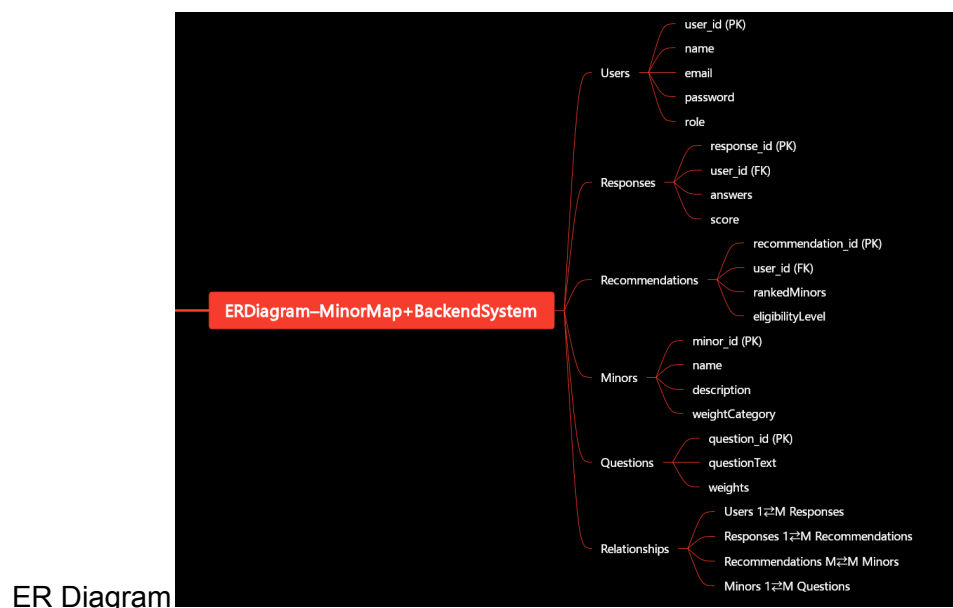
Architecture Diagram



Layer Description

- **Presentation Layer:** HTML/CSS pages for Register, Login, Quiz, and Results.
- **Application Layer:** Express routes, controllers, and middleware handling authentication, quiz logic, and admin actions.
- **Data Layer:** MongoDB collections store structured documents for users, questions, and results.

7. Database Design



ER Diagram

Collections:

- users: { _id, name, email, password, role }
- questions: { _id, questionText, weights: { AI: 2, DS: 3, ... } }
- responses: { _id, userId, answers: [], score }
- recommendations: { _id, userId, rankedMinors: [] }
- minors: { _id, name, description }

Sample Mongoose Schema:

```
const questionSchema = new mongoose.Schema({  
  text: String,  
  weights: { type: Map, of: Number }  
});
```

```
const userSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  password: String,  
  role: { type: String, default: "student" }  
});
```

8. Module Description

Module	Description
1. Authentication Module	Handles user registration, login, password hashing (bcrypt), and JWT-based session management.
2. Quiz Module	Fetches questions and stores user responses in MongoDB.

3. Recommendation Module	Computes eligibility using weighted scoring and returns top 3 minors.
4. Admin Panel Module	Enables CRUD operations for minors, questions, and weight mappings.
5. Middleware & Validation	Manages token verification, error logging, and request validation.
6. API Response Handler	Sends structured JSON responses with proper status codes.

11. Output Screens (UI Only)

Frontend Pages:

- 1. Register Page:**
Simple HTML form for name, email, and password.
- 2. Login Page:**
Email/password input with token storage.
- 3. Select Minor Page:**
Drop-down of available minors before starting the quiz.
- 4. Quiz Page:**
Displays multiple-choice questions fetched from the backend API.
- 5. Results Page:**
Shows recommended minors with labels (Eligible / Potential / Not Recommended).